ANALYSIS I

Question 1:

To simplify financial reports, Amazon India needs to standardise payment values by rounding the average payment values for each payment type.

SQL Query:

```sql
-- Question 1:
-- To simplify financial reports, calculate the average payment value for each payment type,
-- round it to the nearest integer, and display results in ascending order.

SELECT
    payment_type,
    ROUND(AVG(payment_value)) AS rounded_avg_payment
FROM amazon_brazil.payments
GROUP BY payment_type
ORDER BY rounded_avg_payment ASC;
```

Data Output | Messages | Notifications

Showing rows: 1 to 5

	payment_type character varying	rounded_avg_payment numeric
1	not_defined	0
2	voucher	66
3	debit_card	143
4	boleto	145
5	credit_card	163

Explanation:

This query calculates the average payment value for each payment type using the AVG () function. The result is then rounded to the nearest integer using the ROUND () function. The data is grouped by payment type and sorted in ascending order.

Insight:

The analysis shows how different payment methods vary in terms of average transaction value. Payment types with higher average values may indicate preferred methods for high-value purchases. Amazon India can use this insight to optimise payment strategies and offer targeted incentives for specific payment methods.

Question 2:

To refine its payment strategy, Amazon India wants to know the distribution of orders by payment type.

SQL Query:

Query | Query History

```
-- Question 2:
-- Calculate the percentage of total orders for each payment type,
-- rounded to one decimal place, and display in descending order.

SELECT payment_type,
ROUND(COUNT(order_id) * 100.0 / SUM(COUNT(order_id)) OVER (),1) AS percentage_orders
FROM amazon_brazil.payments
GROUP BY payment_type
ORDER BY percentage_orders DESC;
```

Data Output | Messages | Notifications

Showing rows: 1 to 5

	payment_type character varying	percentage_orders numeric
1	credit_card	73.9
2	boleto	19.0
3	voucher	5.6
4	debit_card	1.5
5	not_defined	0.0

Explanation:

This query calculates the percentage contribution of each payment type to total orders. It uses COUNT () to get the number of orders per payment type and a window function SUM(COUNT(...)) OVER () to compute the total number of orders. The result is converted into a percentage and rounded to one decimal place.

Insight:

The distribution highlights the most commonly used payment methods. Payment types with higher percentages indicate customer preference and trust. Amazon India can focus on optimising and promoting these payment methods while improving less-used ones to balance adoption.

Question 3:

Amazon India seeks to create targeted promotions for products within specific price ranges.

SQL Query:

```sql
-- Question 3:
-- Identify all products priced between 100 and 500 BRL that contain the word 'Smart' in their name,
-- and display them sorted by price in descending order.

SELECT
    oi.product_id,
    oi.price
FROM amazon_brazil.order_items oi
JOIN amazon_brazil.products p
    ON oi.product_id = p.product_id
WHERE
    oi.price BETWEEN 100 AND 500
    AND p.product_category_name ILIKE '%smart%'
ORDER BY oi.price DESC;
```

Data Output | Messages | Notifications

	product_id character varying	price numeric
1	1df1a2df8ad2b9d3aa49fd851e314...	439.99
2	1df1a2df8ad2b9d3aa49fd851e314...	439.99
3	7debe59b10825e89c1cbcc8b190c...	349.99
4	7debe59b10825e89c1cbcc8b190c...	349.99
5	ca86b9fe16e12de698c955aedff0ae...	349
6	ca86b9fe16e12de698c955aedff0ae...	349
7	ca86b9fe16e12de698c955aedff0ae...	349
8	ca86b9fe16e12de698c955aedff0ae...	349
9	0e52955ca8143bd179b311cc454a...	335
10	0e52955ca8143bd179b311cc454a...	335
11	7aeaa8f3e592e380c420e8910a717...	329.9
12	7aeaa8f3e592e380c420e8910a717...	329.9
13	7aeaa8f3e592e380c420e8910a717...	329.9
14	7aeaa8f3e592e380c420e8910a717...	329.9
15	7aeaa8f3e592e380c420e8910a717...	329.9
16	7aeaa8f3e592e380c420e8910a717...	329.9
17	7aeaa8f3e592e380c420e8910a717...	329.9
18	7aeaa8f3e592e380c420e8910a717...	329.9
19	7aeaa8f3e592e380c420e8910a717...	329.9
20	7aeaa8f3e592e380c420e8910a717...	329.9
21	7aeaa8f3e592e380c420e8910a717...	329.9

Data Output | Messages | Notifications

	product_id character varying	price numeric
22	7aeaa8f3e592e380c420e8910a717...	329.9
23	d1b571cd58267d8cac8b2afd6e288...	299.9
24	d1b571cd58267d8cac8b2afd6e288...	299.9
25	d1b571cd58267d8cac8b2afd6e288...	299.9
26	d1b571cd58267d8cac8b2afd6e288...	299.9
27	66ffe28d0fd53808d0535eee4b90a...	254
28	66ffe28d0fd53808d0535eee4b90a...	254
29	f06796447de379a26dde5fcac6a1a...	239.9
30	f06796447de379a26dde5fcac6a1a...	239.9
31	d3d5a1d52abe9a7d234908d873fc...	229.9
32	d3d5a1d52abe9a7d234908d873fc...	229.9
33	06ae026e430189633c2fbd0288c86...	217.36
34	06ae026e430189633c2fbd0288c86...	217.36
35	49ef750dc5bf23e3788d4f614bc6d...	198
36	49ef750dc5bf23e3788d4f614bc6d...	198
37	33bb7da523efcdef6cd2996cbf72d...	148
38	33bb7da523efcdef6cd2996cbf72d...	148
39	33bb7da523efcdef6cd2996cbf72d...	148
40	33bb7da523efcdef6cd2996cbf72d...	148
41	6f5795735ab2c629b22669fe889b7...	129.9
42	6f5795735ab2c629b22669fe889b7...	129.9

Data Output | Messages | Notifications

	product_id character varying	price numeric
43	3626035966a7aaee90d68108caeb...	124.9
44	3626035966a7aaee90d68108caeb...	124.9
45	aeaba104830f91586dae1bff90f54a...	123.9
46	aeaba104830f91586dae1bff90f54a...	123.9
47	630c84b1ce83ae0e9ddc05a14103...	110
48	630c84b1ce83ae0e9ddc05a14103...	110
49	3168b2696b15ca440b92afa9e011a...	109.9
50	3168b2696b15ca440b92afa9e011a...	109.9
51	dbd55362ec13c706503b1c71a506...	102
52	dbd55362ec13c706503b1c71a506...	102
53	dbd55362ec13c706503b1c71a506...	102
54	dbd55362ec13c706503b1c71a506...	102
55	dbd55362ec13c706503b1c71a506...	102
56	dbd55362ec13c706503b1c71a506...	102
57	dbd55362ec13c706503b1c71a506...	102
58	dbd55362ec13c706503b1c71a506...	102
59	dbd55362ec13c706503b1c71a506...	102
60	dbd55362ec13c706503b1c71a506...	102
61	dbd55362ec13c706503b1c71a506...	102
62	dbd55362ec13c706503b1c71a506...	102
63	dbd55362ec13c706503b1c71a506...	102

63	dbd55362ec13c706503b1c71a506...	102
64	dbd55362ec13c706503b1c71a506...	102
65	dbd55362ec13c706503b1c71a506...	102
66	dbd55362ec13c706503b1c71a506...	102
67	aeaba104830f91586dae1bff90f54a...	100
68	aeaba104830f91586dae1bff90f54a...	100

Explanation:

This query identifies products priced between 100 and 500 BRL by filtering the order_items table. It joins with the products table to access product category information and filters for products containing the word 'smart'. The results are sorted in descending order of price.

Insight:

Products within this price range that include "smart" in their category indicate potential demand for smart devices. Amazon India can target these products for promotional campaigns to attract tech-savvy customers and increase sales.

Question 4:

To identify seasonal sales patterns, Amazon India wants to determine the top 3 months with the highest total sales.

SQL Query:

Query | Query History

```
83
84  -- Question 4:
85  -- Determine the top 3 months with the highest total sales value,
86  -- rounded to the nearest integer.
87
88  SELECT
89      DATE_TRUNC('month', o.order_purchase_timestamp) AS month,
90      ROUND(SUM(oi.price)) AS total_sales
91  FROM amazon_brazil.orders o
92  JOIN amazon_brazil.order_items oi
93      ON o.order_id = oi.order_id
94  GROUP BY month
95  ORDER BY total_sales DESC
96  LIMIT 3;
```

Data Output | Messages | Notifications

Showing rows: 1 to 3

	month timestamp without time zone	total_sales numeric
1	2017-11-01 00:00:00	2020543
2	2018-04-01 00:00:00	1993296
3	2018-05-01 00:00:00	1993035

Explanation:

This query calculates total monthly sales by joining the orders and order_items tables. The DATE_TRUNC () function is used to group orders by month, and SUM() computes total sales. The results are rounded and sorted in descending order, with the top 3 months selected.

Insight:

The top-performing months indicate peak sales periods, likely driven by seasonal demand, promotions, or holidays. Amazon India can focus marketing efforts and inventory planning around these months to maximize revenue.

Question 5:

Amazon India is interested in product categories with significant price variations.

SQL Query:

Query | Query History

```sql
-- Question 5:
-- Find product categories where the difference between maximum and minimum prices exceeds 500

SELECT
    p.product_category_name,
    MAX(oi.price) - MIN(oi.price) AS price_difference
FROM amazon_brazil.order_items oi
JOIN amazon_brazil.products p
    ON oi.product_id = p.product_id
GROUP BY p.product_category_name
HAVING MAX(oi.price) - MIN(oi.price) > 500
ORDER BY price_difference DESC;
```

	product_category_name character varying	price_difference numeric
1	utilidades_domesticas	6731.94
2	pcs	6694.5
3	artes	6495.5
4	eletroportateis	4792.5
5	instrumentos_musicais	4394.97
6	consoles_games	4094.81
7	esporte_lazer	4054.5
8	relogios_presentes	3990.91
9	[null]	3977
10	ferramentas_jardim	3923.65
11	bebes	3895.46
12	informatica_acessorios	3696.09
13	beleza_saude	3122.8
14	cool_stuff	3102.99
15	construcao_ferramentas_seguranca	3091.0
16	industria_comercio_e_negocios	3061.1
17	agro_industria_e_comercio	2977.01
18	portateis_casa_forno_e_cafe	2888.81
19	pet_shop	2495.1
20	eletronicos	2466.51
21	telefonia	2423

	product_category_name character varying	price_difference numeric
22	eletrodomesticos_2	2336.1
23	construcao_ferramentas_construcao	2299.15
24	automotivo	2254.51
25	eletrodomesticos	2083.81
26	cama_mesa_banho	1992.99
27	market_place	1954.01
28	moveis_decoracao	1894.1
29	construcao_ferramentas_ferramentas	1892.2
30	telefonia_fixa	1784
31	brinquedos	1695.09
32	fashion_bolsas_e_acessorios	1693.99
33	papelaria	1690.71
34	climatizacao	1588.1
35	smart	1444.5
36	dvds_blu_ray	1411.1
37	construcao_ferramentas_jardim	1341.08
38	moveis_cozinha_area_de_servico_jantar_e_jar...	1310.4
39	construcao_ferramentas_iluminacao	1277.49
40	malas_acessorios	1184.57
41	moveis_escritorio	1164.9
42	musica	1162.12

43	casa_construcao	1089.02
44	portateis_cozinha_e_preparadores_de_aliment...	1081.58
45	livros_interesse_geral	893.9
46	tablets_impressao_imagem	875.09
47	cine_foto	867.19
48	moveis_sala	826.09
49	casa_conforto	792.01
50	sinalizacao_e_seguranca	735.5
51	livros_importados	730.01
52	alimentos_bebidas	693.4
53	perfumaria	684.91
54	moveis_quarto	643.1
55	bebidas	617
56	audio	584.09
57	artigos_de_festas	563.21

Explanation:

This query calculates the price difference within each product category by subtracting the minimum price from the maximum price. It joins the order_items and products tables to access both price and category information. Categories with a price difference greater than 500 BRL are filtered using the HAVING clause.

Insight:

Categories with large price variations indicate a wide range of products, from budget to premium. Amazon India can segment these categories further and apply targeted pricing or promotional strategies based on customer segments.

Question 6:

Amazon India wants to identify payment types with the most consistent transaction amounts.

SQL Query:

```sql
-- Question 6:
-- Identify payment types with the least variation in transaction amounts,
-- using standard deviation and sorting by smallest first.

SELECT
    payment_type,
    ROUND(STDDEV(payment_value), 2) AS std_deviation
FROM amazon_brazil.payments
GROUP BY payment_type
ORDER BY std_deviation ASC;
```

Data Output | Messages | Notifications

Showing rows: 1 to 5

	payment_type character varying	std_deviation numeric
1	not_defined	0.00
2	voucher	115.52
3	boleto	213.58
4	credit_card	222.12
5	debit_card	245.79

Explanation:

This query calculates the standard deviation of payment values for each payment type using the STDDEV () function. Standard deviation measures the variation or dispersion in values. The results are rounded and sorted in ascending order to identify the most consistent payment types.

Insight:

Payment types with lower standard deviation indicate more consistent transaction values, which may reflect stable customer behaviour. Amazon India can prioritise these payment methods to ensure predictable revenue streams and optimise pricing or promotions accordingly.

Question 7:

Amazon India wants to identify products that may have incomplete names to fix data quality issues.

SQL Query:

```
Query  Query History
122  -- Question 7:
123  -- Retrieve products where the product category name is missing
124  -- or contains only a single character.
125
126  SELECT
127      product_id,
128      product_category_name
129  FROM amazon_brazil.products
130  WHERE
131      product_category_name IS NULL
132      OR LENGTH(product_category_name) = 1
133      LIMIT 10;
134
```

	product_id [PK] character varying	product_category_name character varying
1	a41e356c76fab66334f36de622ecbd...	[null]
2	d8dee61c2034d6d075997acef1870...	[null]
3	56139431d72cd51f19eb9f7dae4d16...	[null]
4	46b48281eb6d663ced748f324108c...	[null]
5	5fb61f482620cb672f5e586bb132ea...	[null]
6	e10758160da97891c2fdcbc35f0f03...	[null]
7	39e3b9b12cd0bf8ee681bbc1c130fe...	[null]
8	794de06c32a626a5692ff50e4985d3...	[null]
9	7af3e2da474486a3519b0cba9dea8...	[null]
10	629beb8e7317703dcc5f35b5463fd2...	[null]

Note: *The full result contains 614 rows. Only a sample of 10 rows is displayed*

Explanation:

This query retrieves products where the category name is either missing (NULL) or contains only a single character. The LENGTH () function is used to identify unusually short category names, which may indicate incomplete or incorrect data.

Insight:

Products with missing or incomplete category names indicate data quality issues. Amazon India should clean and standardize this data to improve product categorization, search accuracy, and overall customer experience.

ANALYSIS II

Question 1:

Amazon India wants to understand which payment types are most popular across different order value segments.

SQL Query:

```sql
-- Part II - Question 1:
-- Segment orders into value ranges and calculate the count of each payment type within those segments.

SELECT
    CASE
        WHEN oi.price < 200 THEN 'Low'
        WHEN oi.price BETWEEN 200 AND 1000 THEN 'Medium'
        ELSE 'High'
    END AS order_value_segment,
    p.payment_type,
    COUNT(*) AS count
FROM amazon_brazil.order_items oi
JOIN amazon_brazil.payments p
    ON oi.order_id = p.order_id
GROUP BY order_value_segment, p.payment_type
ORDER BY count DESC;
```

	order_value_segment text	payment_type character varying	count bigint
1	Low	credit_card	150996
2	Low	boleto	41598
3	Medium	credit_card	21086
4	Low	voucher	11330
5	Medium	boleto	3908
6	Low	debit_card	3070
7	High	credit_card	1456
8	Medium	voucher	1150
9	Medium	debit_card	286
10	High	boleto	228
11	High	voucher	68
12	High	debit_card	26

Explanation:

This query segments orders into three price-based value categories using a CASE statement. It then joins payment data to identify the payment type used and counts occurrences within each segment. The results are grouped and sorted in descending order of count.

Insight:

The analysis reveals which payment methods dominate in low, medium, and high-value transactions. Amazon India can tailor payment offers or incentives based on customer behaviour within each segment to improve conversion rates.

Question 2:

Amazon India wants to analyse the price range and average price for each product category.

SQL Query:

```sql
-- Part II - Question 2:
-- Calculate minimum, maximum, and average price for each product category,
-- and sort results by average price in descending order.

SELECT
    p.product_category_name,
    MIN(oi.price) AS min_price,
    MAX(oi.price) AS max_price,
    ROUND(AVG(oi.price), 2) AS avg_price
FROM amazon_brazil.order_items oi
JOIN amazon_brazil.products p
    ON oi.product_id = p.product_id
GROUP BY p.product_category_name
ORDER BY avg_price DESC
LIMIT 10;
```

	product_category_name character varying	min_price numeric	max_price numeric	avg_price numeric
1	pcs	34.5	6729	1098.34
2	portateis_casa_forno_e_cafe	10.19	2899	624.29
3	eletrodomesticos_2	13.9	2350	476.12
4	agro_industria_e_comercio	12.99	2990	341.66
5	instrumentos_musicais	4.9	4399.87	281.62
6	eletroportateis	6.5	4799	280.78
7	portateis_cozinha_e_preparadores_de_alimen...	17.42	1099	264.57
8	telefonia_fixa	6	1790	225.69
9	construcao_ferramentas_seguranca	8.9	3099.9	208.99
10	relogios_presentes	8.99	3999.9	200.91

Explanation:

This query calculates the minimum, maximum, and average price for each product category by joining the order_items and products tables. It groups results by category and sorts them in descending order based on the average price.

Insight:

Categories with higher average prices represent premium segments, while those with lower averages indicate budget-friendly products. Amazon India can use this information to tailor pricing strategies and marketing campaigns for different customer segments.

Question 3:

Amazon India wants to identify customers who have placed multiple orders over time.

SQL Query:

```sql
-- Part II - Question 3:
-- Identify customers who have placed more than one order and display their total order count.

SELECT
    c.customer_unique_id,
    COUNT(o.order_id) AS total_orders
FROM amazon_brazil.orders o
JOIN amazon_brazil.customers c
    ON o.customer_id = c.customer_id
GROUP BY c.customer_unique_id
HAVING COUNT(o.order_id) > 1
ORDER BY total_orders DESC
LIMIT 10;
```

	customer_unique_id character varying	total_orders bigint
1	5bf4ea2d98005b960eea0dbf652ef...	16
2	6af40347f5dd7bdd65437a35e1b2f...	16
3	3d364a7768fae99678635c4370295...	16
4	417b909c0962b2610f1cfeb1c1478...	16
5	0fdc0d21e1983e8af4d399e17671f...	16
6	5f94af52aef02c968a2e0f01f43086...	16
7	1b6d29725255a77667a8c639eeb4...	16
8	363f980585bf04c1a88fdb986011c...	16
9	4034aa08d48695a538b7030910aa...	16
10	75f15790b1852b42b1dbf645d98ffa...	16

Note: *Only a sample of top 10 customers is shown. The full result contains 3140 rows.*

Explanation:

This query joins the orders and customers tables to link orders with customer identities. It groups data by customer and counts the number of orders per customer. The HAVING clause filters customers with more than one order.

Insight:

Customers with multiple orders represent loyal or repeat buyers. Amazon India can target these customers with loyalty programs, personalised offers, and retention strategies to increase long-term engagement.

Question 4:

Amazon India wants to categorize customers based on their purchase history.

SQL Query:

```sql
-- Part II - Question 4:
-- Categorize customers into New, Returning, and Loyal based on their total order count.

WITH customer_orders AS (
    SELECT
        c.customer_unique_id,
        COUNT(o.order_id) AS total_orders
    FROM amazon_brazil.orders o
    JOIN amazon_brazil.customers c
        ON o.customer_id = c.customer_id
    GROUP BY c.customer_unique_id
)
SELECT
    customer_unique_id,
    CASE
        WHEN total_orders = 1 THEN 'New'
        WHEN total_orders BETWEEN 2 AND 4 THEN 'Returning'
        ELSE 'Loyal'
    END AS customer_type
FROM customer_orders
LIMIT 10;
```

	customer_unique_id character varying	customer_type text
1	0000366f3b9a7992bf8c76cfdf3221...	New
2	0000b849f77a49e4a4ce2b2a4ca5b...	New
3	0000f46a3911fa3c0805444483337...	New
4	0000f6ccb0745a6a4b88665a16c9f...	New
5	0004aac84e0df4da2b147fca70cf82...	New
6	0004bd2a26a76fe21f786e4fbd8060...	New
7	00050ab1314c0e55a6ca13cf7181fe...	New
8	00053a61a98854899e70ed204dd4b...	New
9	0005e1862207bf6ccc02e4228effd9...	New
10	0005ef4cd20d2893f0d9fbd94d3c0d...	New

Note: *Only a sample of 10 rows is shown. The full dataset includes all categorised customers, with a total of 95077.*

Explanation:

This query first calculates the total number of orders per customer using a Common Table Expression (CTE). It then categorises customers into New, Returning, and Loyal segments using a CASE statement based on their order count.

Insight:

Customer segmentation helps identify different behaviour patterns. Loyal customers can be targeted with premium offers, while returning customers can be nurtured into loyal ones through engagement strategies.

Question 5:

Amazon India wants to identify which product categories generate the most revenue.

SQL Query:

```sql
-- Part II - Question 5:
-- Calculate total revenue for each product category and display the top 5 categories.

SELECT
    p.product_category_name,
    ROUND(SUM(oi.price), 2) AS total_revenue
FROM amazon_brazil.order_items oi
JOIN amazon_brazil.products p
    ON oi.product_id = p.product_id
GROUP BY p.product_category_name
ORDER BY total_revenue DESC
LIMIT 5;
```

Data Output | Messages | Notifications

Showing rows: 1 to 5

	product_category_name character varying	total_revenue numeric
1	beleza_saude	2515730.68
2	relogios_presentes	2406120.64
3	cama_mesa_banho	2064537.18
4	esporte_lazer	1971762.20
5	informatica_acessorios	1821210.14

***Note**: The complete analysis includes 79 product categories. Only the top 5 categories based on total revenue are displayed as per the requirement.*

Explanation:

This query calculates total revenue for each product category by summing up prices from the order_items table. It joins with the products table to get category information, groups results by category and selects the top 5 categories based on revenue.

Insight:

The top-performing categories contribute the most to overall revenue. Amazon India should prioritise these categories for marketing campaigns, inventory optimisation, and strategic investments to maximise profitability.

ANALYSIS III

Question 1:

The marketing team wants to compare total sales across different seasons.

SQL Query:

Query | Query History

```
210
211  -- Part III - Question 1:
212  -- Calculate total sales for each season based on order purchase dates.
213
214  SELECT
215      season,
216      ROUND(SUM(price), 2) AS total_sales
217  FROM (
218      SELECT
219          oi.price,
220          CASE
221              WHEN EXTRACT(MONTH FROM o.order_purchase_timestamp) IN (3, 4, 5) THEN 'Spring'
222              WHEN EXTRACT(MONTH FROM o.order_purchase_timestamp) IN (6, 7, 8) THEN 'Summer'
223              WHEN EXTRACT(MONTH FROM o.order_purchase_timestamp) IN (9, 10, 11) THEN 'Autumn'
224              ELSE 'Winter'
225          END AS season
226      FROM amazon_brazil.orders o
227      JOIN amazon_brazil.order_items oi
228          ON o.order_id = oi.order_id
229  ) AS seasonal_data
230  GROUP BY season
231  ORDER BY total_sales DESC;
232
```

Data Output | Messages | Notifications

	season text	total_sales numeric
1	Spring	8433443.08
2	Summer	8240719.24
3	Winter	5811500.06
4	Autumn	4697625.02

Explanation:

This query uses a subquery to assign each order to a season based on its purchase month using a CASE statement. It then calculates total sales for each season by summing the price values and grouping by season.

Insight:

Seasonal sales patterns can help Amazon India identify peak demand periods. The company can align marketing campaigns, promotions, and inventory planning based on high-performing seasons to maximize sales.

Question 2:

The inventory team wants to identify products with sales volumes above the overall average.

SQL Query:

```sql
-- Part III - Question 2:
-- Identify products whose total quantity sold is above the overall average.

SELECT
    product_id,
    COUNT(order_item_id) AS total_quantity_sold
FROM amazon_brazil.order_items
GROUP BY product_id
HAVING COUNT(order_item_id) > (
    SELECT AVG(product_count)
    FROM (
        SELECT COUNT(order_item_id) AS product_count
        FROM amazon_brazil.order_items
        GROUP BY product_id
    ) AS avg_table
);
```

	product_id character varying	total_quantity_sold bigint
1	001795ec6f1b187d37335e1c47047...	18
2	001b72dfd63e9833e8c02742adf472...	28
3	00210e41887c2a8ef9f791ebc780cc...	14
4	002159fe700ed3521f46cfcf6e941c76	16
5	00250175f79f584c14ab5cecd80553...	22
6	002af88741ba70c7b5cf4e4a0ad7ef...	12
7	004636c889c7c3dad6631f136b7fa0...	8
8	005030ef108f58b46b78116f754d8d...	26
9	007c63ae4b346920756b5adcad809...	12
10	00878d953636afec00d3e85d55a12...	22

***Note**: Only a sample of 10 products is displayed. The full result includes all products with above-average sales.*

Explanation:

This query calculates the total quantity sold for each product using COUNT (). It then compares each product's quantity with the overall average quantity, calculated using a subquery. Only products with above-average sales are selected.

Insight:

Products with higher-than-average sales indicate strong demand. Amazon India can prioritize these

Question 3:

The finance team wants to analyze monthly revenue trends for the year 2018.

SQL Query:

```sql
-- Part III - Question 3:
-- Calculate total monthly revenue for the year 2018.

SELECT
    DATE_TRUNC('month', o.order_purchase_timestamp) AS month,
    ROUND(SUM(oi.price), 2) AS total_revenue
FROM amazon_brazil.orders o
JOIN amazon_brazil.order_items oi
    ON o.order_id = oi.order_id
WHERE EXTRACT(YEAR FROM o.order_purchase_timestamp) = 2018
GROUP BY month
ORDER BY month;
```

	month timestamp without time zone	total_revenue numeric
1	2018-01-01 00:00:00	1900060.72
2	2018-02-01 00:00:00	1688357.42
3	2018-03-01 00:00:00	1966426.88
4	2018-04-01 00:00:00	1993295.50
5	2018-05-01 00:00:00	1993035.36
6	2018-06-01 00:00:00	1730248.62
7	2018-07-01 00:00:00	1791014.44
8	2018-08-01 00:00:00	1709372.66
9	2018-09-01 00:00:00	290.00

Explanation:

This query calculates total revenue for each month in 2018 by joining orders and order items. It groups data by month using DATE_TRUNC () and sums the price values.

Insight:

The trend highlights peak and low sales periods across the year. Amazon India can use this information to plan promotions, optimize inventory, and align marketing efforts with seasonal demand patterns.

products for inventory restocking, promotions, and supply chain optimization.

Question 4:

Amazon India is designing a loyalty program and wants to segment customers based on purchase frequency.

SQL Query:

```sql
-- Part III - Question 4:
-- Segment customers based on purchase frequency and count the number of customers in each segment.

WITH customer_orders AS (
    SELECT
        c.customer_unique_id,
        COUNT(o.order_id) AS total_orders
    FROM amazon_brazil.orders o
    JOIN amazon_brazil.customers c
        ON o.customer_id = c.customer_id
    GROUP BY c.customer_unique_id
),
categorized AS (
    SELECT
        CASE
            WHEN total_orders BETWEEN 1 AND 2 THEN 'Occasional'
            WHEN total_orders BETWEEN 3 AND 5 THEN 'Regular'
            ELSE 'Loyal'
        END AS customer_type
    FROM customer_orders
)

SELECT
    customer_type,
    COUNT(*) AS count
FROM categorized
GROUP BY customer_type
ORDER BY count DESC;
```

Data Output | Messages | Notifications

	customer_type text	count bigint
1	Occasional	94635
2	Regular	333
3	Loyal	109

Explanation:

This query uses a CTE to calculate total orders per customer and then categorizes them into Occasional, Regular, and Loyal segments using a CASE statement. It then counts the number of customers in each segment.

Insight:

Customer segmentation reveals the distribution of purchase behavior. Amazon India can target Loyal customers with rewards, encourage Regular customers to increase purchases, and convert Occasional buyers into repeat customers.

Question 5:

Amazon wants to identify high-value customers to target for an exclusive rewards program.

SQL Query:

Query | Query History

```sql
-- Part III - Question 5:
-- Rank customers based on their average order value and display the top 20.

WITH order_values AS (
    -- Total value per order
    SELECT
        o.order_id,
        o.customer_id,
        SUM(oi.price) AS order_total
    FROM amazon_brazil.orders o
    JOIN amazon_brazil.order_items oi
        ON o.order_id = oi.order_id
    GROUP BY o.order_id, o.customer_id
),
customer_avg AS (
    -- Average order value per customer
    SELECT
        c.customer_id,
        AVG(ov.order_total) AS avg_order_value
    FROM order_values ov
    JOIN amazon_brazil.customers c
        ON ov.customer_id = c.customer_id
    GROUP BY c.customer_id
)
SELECT
    customer_id,
    ROUND(avg_order_value, 2) AS avg_order_value,
    RANK() OVER (ORDER BY avg_order_value DESC) AS customer_rank
FROM customer_avg
ORDER BY customer_rank
LIMIT 20;
```

SQL

	customer_id [PK] character varying	avg_order_value numeric	customer_rank bigint
1	1617b1357756262bfa56ab541c47b...	26880.00	1
2	ec5b2ba62e574342386871631fafd...	14320.00	2
3	c6e2731c5b391845f6800c97401a4...	13470.00	3
4	f48d464a0baaea338cb25f816991ab...	13458.00	4
5	3fd6777bbce08a352fddd04e4a7cc8...	12998.00	5
6	05455dfa7cd02f13d132aa7a6a972...	11869.20	6
7	df55c14d1476a9a3467f131269c24...	9598.00	7
8	24bbf5fd2f2e1b359ee7de94defc4a15	9380.00	8
9	e0a2412720e9ea4f26c1ac985f6a73...	9199.80	9
10	3d979689f636322c62418b6346b1c...	9180.00	10
11	cc803a2c412833101651d3f90ca7d...	8800.00	11
12	1afc82cd60e303ef09b4ef9837c950...	8799.74	12
13	35a413c7ca3c69756cb75867d6311...	8199.98	13
14	e9b0d0eb3015ef1c9ce6cf5b9dcbee...	8118.00	14
15	c6695e3b1e48680db36b487419fb0...	7999.80	15
16	926b6a6fb8b6081e00b335edaf578...	7998.00	16
17	3be2c536886b2ea4668eced3a80dd...	7960.00	17
18	31e83c01fce824d0ff786fcd48dad009	7860.00	18
19	eb7a157e8da9c488cd4ddc48711f1...	7798.00	19
20	19b32919fa1198aefc0773ee2e46e6...	7400.00	20

Explanation:

This query first calculates the total order value per order using a CTE. It then computes the average order value per customer and ranks customers using a window function (RANK ()) based on this value. The top 20 customers are selected.

Insight:

Customers with the highest average order values represent premium buyers. Amazon India can target these customers with exclusive rewards, personalised offers, and loyalty benefits to increase retention and lifetime value.

Question 6:

Amazon wants to analyse sales growth trends for key products over time.

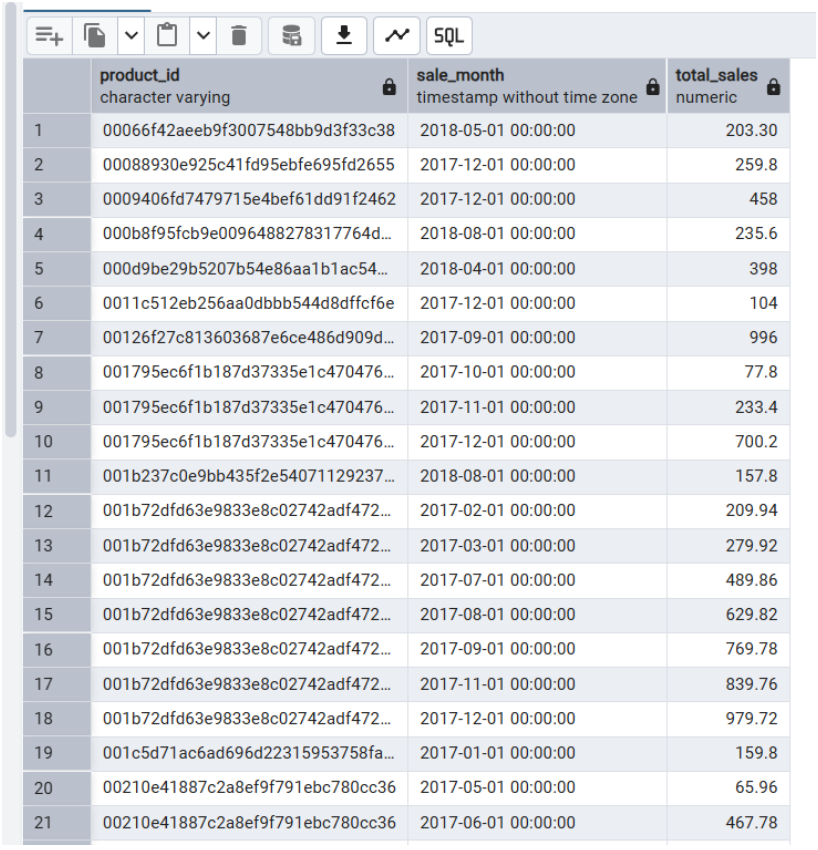
SQL Query:

```sql
-- Part III - Question 6:
-- Calculate monthly cumulative sales for each product.

WITH monthly_sales AS (
    SELECT
        oi.product_id,
        DATE_TRUNC('month', o.order_purchase_timestamp) AS sale_month,
        SUM(oi.price) AS monthly_total
    FROM amazon_brazil.orders o
    JOIN amazon_brazil.order_items oi
        ON o.order_id = oi.order_id
    GROUP BY oi.product_id, sale_month
)

SELECT
    product_id,
    sale_month,
    SUM(monthly_total) OVER (
        PARTITION BY product_id
        ORDER BY sale_month
    ) AS total_sales
FROM monthly_sales
ORDER BY product_id, sale_month;

```



	product_id character varying	sale_month timestamp without time zone	total_sales numeric
1	00066f42aeeb9f3007548bb9d3f33c38	2018-05-01 00:00:00	203.30
2	00088930e925c41fd95ebfe695fd2655	2017-12-01 00:00:00	259.8
3	0009406fd7479715e4bef61dd91f2462	2017-12-01 00:00:00	458
4	000b8f95fcb9e0096488278317764d...	2018-08-01 00:00:00	235.6
5	000d9be29b5207b54e86aa1b1ac54...	2018-04-01 00:00:00	398
6	0011c512eb256aa0dbbb544d8dffcf6e	2017-12-01 00:00:00	104
7	00126f27c813603687e6ce486d909d...	2017-09-01 00:00:00	996
8	001795ec6f1b187d37335e1c470476...	2017-10-01 00:00:00	77.8
9	001795ec6f1b187d37335e1c470476...	2017-11-01 00:00:00	233.4
10	001795ec6f1b187d37335e1c470476...	2017-12-01 00:00:00	700.2
11	001b237c0e9bb435f2e54071129237...	2018-08-01 00:00:00	157.8
12	001b72dfd63e9833e8c02742adf472...	2017-02-01 00:00:00	209.94
13	001b72dfd63e9833e8c02742adf472...	2017-03-01 00:00:00	279.92
14	001b72dfd63e9833e8c02742adf472...	2017-07-01 00:00:00	489.86
15	001b72dfd63e9833e8c02742adf472...	2017-08-01 00:00:00	629.82
16	001b72dfd63e9833e8c02742adf472...	2017-09-01 00:00:00	769.78
17	001b72dfd63e9833e8c02742adf472...	2017-11-01 00:00:00	839.76
18	001b72dfd63e9833e8c02742adf472...	2017-12-01 00:00:00	979.72
19	001c5d71ac6ad696d22315953758fa...	2017-01-01 00:00:00	159.8
20	00210e41887c2a8ef9f791ebc780cc36	2017-05-01 00:00:00	65.96
21	00210e41887c2a8ef9f791ebc780cc36	2017-06-01 00:00:00	467.78

Note: Only a sample of results is displayed. The full dataset (62176 rows) includes cumulative monthly sales for all products.

Explanation:

This query first calculates monthly sales for each product using a CTE. It then computes cumulative sales over time using a window function (SUM OVER) partitioned by product and ordered by month.

Insight:

Cumulative sales trends help identify product growth patterns. Amazon India can use this data to track product lifecycle performance and make informed decisions about promotions, inventory, and product strategy.

Question 7:

Amazon wants to analyse how different payment methods affect monthly sales growth.

SQL Query:

```sql
-- Part III - Question 7:
-- Calculate monthly sales and month-over-month growth for each payment type in 2018.

WITH monthly_sales AS (
    SELECT
        p.payment_type,
        DATE_TRUNC('month', o.order_purchase_timestamp) AS sale_month,
        SUM(oi.price) AS monthly_total
    FROM amazon_brazil.orders o
    JOIN amazon_brazil.order_items oi
        ON o.order_id = oi.order_id
    JOIN amazon_brazil.payments p
        ON o.order_id = p.order_id
    WHERE EXTRACT(YEAR FROM o.order_purchase_timestamp) = 2018
    GROUP BY p.payment_type, sale_month
)
SELECT
    payment_type,
    sale_month,
    ROUND(monthly_total, 2) AS monthly_total,
    ROUND(
        (monthly_total - LAG(monthly_total) OVER (
            PARTITION BY payment_type
            ORDER BY sale_month
        ))
        * 100.0
         / LAG(monthly_total) OVER (
             PARTITION BY payment_type
             ORDER BY sale_month
         ),
         2
    ) AS monthly_change
FROM monthly_sales
ORDER BY payment_type, sale_month;
```

Data Output | Messages | Notifications

	payment_type character varying	sale_month timestamp without time zone	monthly_total numeric	monthly_change numeric
1	boleto	2018-01-01 00:00:00	341302.80	[null]
2	boleto	2018-02-01 00:00:00	306331.54	-10.25
3	boleto	2018-03-01 00:00:00	315614.76	3.03
4	boleto	2018-04-01 00:00:00	325881.22	3.25
5	boleto	2018-05-01 00:00:00	333143.20	2.23
6	boleto	2018-06-01 00:00:00	252759.80	-24.13
7	boleto	2018-07-01 00:00:00	325876.84	28.93
8	boleto	2018-08-01 00:00:00	236428.24	-27.45
9	credit_card	2018-01-01 00:00:00	1520505.52	[null]
10	credit_card	2018-02-01 00:00:00	1360397.98	-10.53
11	credit_card	2018-03-01 00:00:00	1627130.50	19.61
12	credit_card	2018-04-01 00:00:00	1637060.44	0.61
13	credit_card	2018-05-01 00:00:00	1632976.22	-0.25
14	credit_card	2018-06-01 00:00:00	1420480.86	-13.01
15	credit_card	2018-07-01 00:00:00	1390161.10	-2.13
16	credit_card	2018-08-01 00:00:00	1391561.34	0.10
17	debit_card	2018-01-01 00:00:00	19351.98	[null]
18	debit_card	2018-02-01 00:00:00	12178.12	-37.07
19	debit_card	2018-03-01 00:00:00	13789.76	13.23
20	debit_card	2018-04-01 00:00:00	18095.26	31.22
21	debit_card	2018-05-01 00:00:00	17585.68	-2.82
22	debit_card	2018-06-01 00:00:00	62668.88	256.36
23	debit_card	2018-07-01 00:00:00	72657.98	15.94
24	debit_card	2018-08-01 00:00:00	79561.22	9.50
25	voucher	2018-01-01 00:00:00	93142.52	[null]
26	voucher	2018-02-01 00:00:00	81291.94	-12.72
27	voucher	2018-03-01 00:00:00	95337.10	17.28
28	voucher	2018-04-01 00:00:00	77907.68	-18.28
29	voucher	2018-05-01 00:00:00	80595.78	3.45
30	voucher	2018-06-01 00:00:00	82532.66	2.40
31	voucher	2018-07-01 00:00:00	61909.00	-24.99
32	voucher	2018-08-01 00:00:00	61764.00	-0.23
33	voucher	2018-09-01 00:00:00	290.00	-99.53

Explanation:

This query calculates total monthly sales for each payment type in 2018 using a CTE. It then computes the month-over-month growth using the LAG () window function to compare current and previous month sales.

Note: Only a sample of results is displayed. The full dataset includes monthly growth trends for all payment types.

Insight:

Understanding payment-wise growth trends helps identify which payment methods drive sales increases. Amazon India can promote high-growth payment methods and improve underperforming ones.